

GraphQL Blog — Authentication and Authorization

Project: GraphQL Blog

Technology: Python, FastAPI, Strawberry GraphQL, SQLAlchemy, PostgreSQL

Location: /home/dominic/Documents/graphql-blog

Report #: 6/20

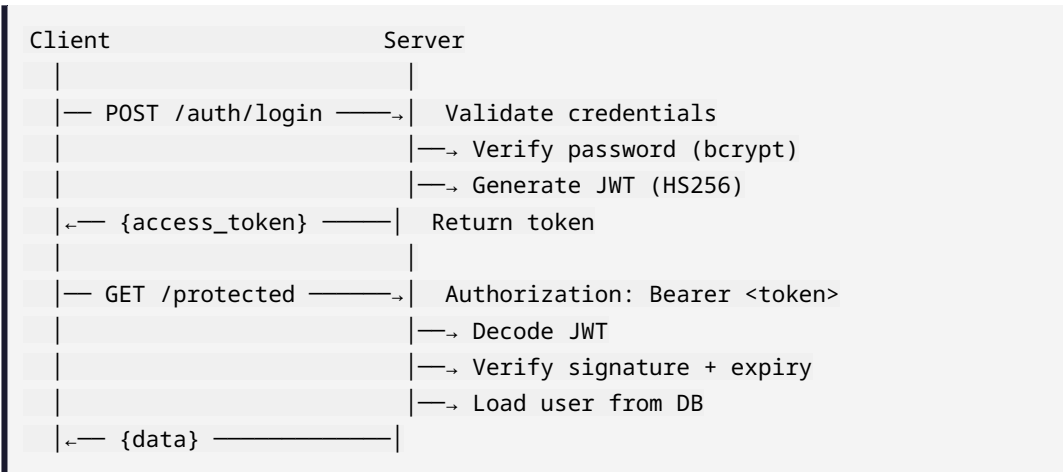
JWT Authentication

Uses `python-jose` with HS256 algorithm. Token includes `sub` (user ID).

Protected: Post mutations (create, update, delete) **Open:** Registration, login, public post queries **Password storage:** bcrypt via passlib

Authentication & Authorization System

Auth Flow



JWT Configuration

Parameter	Value	Notes
Algorithm	HS256	Symmetric signing
Expiry	30 days	Configurable via ACCESS_TOKEN_EXPIRE
Header	Authorization: Bearer	Standard Bearer scheme
Payload	{"sub": user_id, "exp": timestamp}	Standard JWT claims

Password Security

- Hashing algorithm: bcrypt (via passlib)
- Salt: automatic (bcrypt built-in)
- Minimum password length: 8 characters
- No plaintext storage ever

Protected Routes

- POST /urls/shorten (JWT required)
- GET /urls/list (JWT required)
- DELETE /urls/{code} (JWT + ownership)
- POST /api/chat/message (JWT required)
- POST /api/rag/upload (JWT required)

Permission Model

- Resource ownership: users can only modify their own resources
- No role-based access currently (admin/user distinction not implemented)
- Future: add is_admin flag for elevated permissions

Security Headers (Recommended)

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevent MIME sniffing
X-Frame-Options	DENY	Prevent clickjacking
Strict-Transport-Security	max-age=31536000	Force HTTPS